Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Data Visualization with Power BI and Tableau

TY BScIT – AI MINI PROJECT

EMERGENCY COMPLAINT PRIORITY SYSTEM

Artificial Intelligence

Particular	Details
Student Name	Roshni Shahu
Roll Number	T041
Class	TYIT (5th Semester)
College Name	MVLU
Academic Year	2026–27
Subject	Artificial Intelligence (AI)

Submitted as part of the AI Mini Project

Contents

1. 1. Cover Page
2. 2. Introduction
3. 3. Problem Statement
4. 4. Objectives
5. 5. Technologies Used
6. 6. Dataset
7. 7. Methodology / Working
8. 8. Algorithm Used
9. 9. Implementation
10. 10. Results
11. 11. Advantages
12. 12. Limitations
13. 13. Applications
14. 14. Conclusion
15. 15. Future Scope
16. 16. References

Project Summary

The Emergency Complaint Priority System is a machine-learning based web application that predicts the priority of a complaint as High, Medium, or Low. The system uses complaint category, severity, number of people affected, and response urgency as input features.

2. Introduction

The Emergency Complaint Priority System is an AI-based application designed to help organizations handle complaints according to their urgency. In real-life situations, a large number of complaints may be received at the same time, but every complaint does not require the same response level.

The project uses a Decision Tree Classifier to learn patterns from a complaint dataset. When a user enters the details of a new complaint, the trained model predicts whether the complaint should be treated as High, Medium, or Low priority.

The project is useful because it can reduce manual sorting work, support faster decision-making, and help staff identify complaints that require immediate attention.

How the project solves the problem

- Collects complaint-related information in a structured form.
- Uses a trained machine-learning model to classify complaint priority.
- Displays the predicted priority through a simple web interface.
- Can support organizations in arranging complaints according to urgency.

3. Problem Statement

Many organizations receive a large number of complaints every day, but all complaints do not have the same level of urgency. Manually checking and prioritizing every complaint can take time and may cause important emergency complaints to be delayed. The proposed Emergency Complaint Priority System uses a Decision Tree Machine Learning algorithm to automatically classify complaints into High, Medium, or Low priority based on complaint category, severity, number of people affected, and response urgency.

4. Objectives

- To collect and prepare a complaint-priority dataset.
- To preprocess categorical and numerical data for machine learning.
- To apply a Decision Tree Classifier to the complaint-priority problem.
- To train and test the machine-learning model and measure its performance.
- To develop a simple web interface that generates complaint priority predictions.

5. Technologies Used

Technology	Purpose
Python	Programming language used for data processing, machine learning, and backend logic.
Pandas	Used to load, inspect, and process the CSV dataset.
Scikit-learn	Used for Label Encoding, train-test splitting, Decision Tree training, and accuracy evaluation.
Flask	Used to create the web application and connect the user interface with the trained model.

HTML	Used to create the complaint input form.
CSS	Used to design and format the web interface.
Pickle	Used to save and load the trained machine-learning model and encoders.
VS Code	Development environment used to create and run the project.

6. Dataset

Dataset Name: emergency_complaints_dataset.csv

Source of Dataset: Synthetic educational dataset created specifically for this AI mini project.

Number of Records: 300 records.

Number of Attributes: 5 columns including four input features and one target variable.

Important Attributes / Features

Attribute	Description	Role
complaint_category	Type of complaint such as Fire, Medical, Accident, Water, Garbage, etc.	Input
severity	Severity level from 1 to 5.	Input
people_affected	Approximate number of people affected by the complaint.	Input
response_urgency	Urgency of response from 1 to 5.	Input
priority	Final complaint priority: High, Medium, or Low.	Target

Sample Dataset Records

complaint_category	severity	people_affected	response_urgency	priority
Noise	1	2	3	Low
Security	2	9	1	Low
Noise	5	6	5	Medium
Gas Leak	1	2	1	Low
Security	2	33	5	High
Fire	5	13	5	High
Gas Leak	2	29	5	High
Electricity	1	49	2	Low
Drainage	4	22	3	Medium
Accident	2	49	3	Medium

Note: The dataset is synthetic and intended for academic demonstration. It does not represent actual emergency-department records or real-world operational decisions.

7. Methodology / Working

The project follows these steps:

17. Load the complaint dataset using Pandas.
18. Inspect the dataset and identify the input features and target variable.
19. Convert complaint categories and priority labels into numerical values using LabelEncoder.
20. Separate the input features (X) and target variable (y).
21. Split the dataset into 80% training data and 20% testing data.
22. Create a Decision Tree Classifier.
23. Train the model using the training data.
24. Predict priorities for the testing data.
25. Calculate the model accuracy and display a classification report.
26. Save the trained model and encoders in priority_model.pkl.
27. Load the saved model through Flask.
28. Accept new complaint details from the web form and display the predicted priority.

Working Flow

Stage	Flow	Next Stage
Complaint Dataset	↓	Pandas Data Loading
Pandas Data Loading	↓	Preprocessing / Encoding
Preprocessing / Encoding	↓	Train-Test Split
Train-Test Split	↓	Decision Tree Training
Decision Tree Training	↓	Model Testing
Model Testing	↓	Saved Model
Saved Model	↓	Flask Web Application
Flask Web Application	↓	High / Medium / Low Prediction

8. Algorithm Used

Decision Tree Classifier

A Decision Tree is a supervised machine-learning algorithm used for classification and regression. It makes decisions by splitting data into branches based on feature values and finally reaches a prediction at a leaf node.

Why Decision Tree is used

- The project requires classification into three clear categories: High, Medium, and Low.
- It works with both numerical features and encoded categorical data.
- The decision-making structure is relatively easy to understand and explain.
- It is suitable for a small academic classification project.

How it works in this project

29. The model receives encoded complaint category, severity, people affected, and response urgency.
30. The tree learns relationships between these features and the known priority labels.
31. During prediction, the new complaint follows decision branches based on its feature values.

32. The final leaf provides the predicted priority.

9. Implementation

The implementation is divided into model training and web application development.

9.1 Model Training – train_model.py

```
import pandas as pd
import pickle

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv("emergency_complaints_dataset.csv")

category_encoder = LabelEncoder()
priority_encoder = LabelEncoder()

data["complaint_category"] = category_encoder.fit_transform(
    data["complaint_category"]
)

data["priority"] = priority_encoder.fit_transform(
    data["priority"]
)

X = data[
    ["complaint_category", "severity",
     "people_affected", "response_urgency"]
]

y = data["priority"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

model = DecisionTreeClassifier(
    random_state=42, max_depth=5
)

model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Model Training Completed")
print("Accuracy:", accuracy * 100, "%")

print(classification_report(
    y_test,
    y_pred,
    target_names=priority_encoder.classes_,
    zero_division=0
))

with open("priority_model.pkl", "wb") as file:
    pickle.dump(
        (model, category_encoder, priority_encoder),
        file
    )
```

Code Explanation

- Pandas loads the CSV dataset.
- LabelEncoder converts text categories into numerical values.
- train_test_split creates training and testing subsets.
- DecisionTreeClassifier trains the classification model.
- accuracy_score measures the prediction accuracy.
- Pickle saves the trained model and encoders for use by the Flask application.

9.2 Flask Web Application – app.py

```
from flask import Flask, render_template, request
import pickle

app = Flask(__name__)

with open("priority_model.pkl", "rb") as file:
    model, category_encoder, priority_encoder = pickle.load(file)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():

    category = request.form["category"]
```

```
    severity = int(request.form["severity"])
    people = int(request.form["people"])
    response = int(request.form["response"])

    category_encoded = category_encoder.transform(
        [category]
    )[0]

    input_data = [[
        category_encoded,
        severity,
        people,
        response
    ]]

    prediction = model.predict(input_data)

    priority = priority_encoder.inverse_transform(
        prediction
    )[0]

    return render_template(
        "index.html",
        result=priority
    )

if __name__ == "__main__":
    app.run(debug=True)
```

9.3 Web Interface

The HTML page provides fields for complaint category, severity, number of people affected, and response urgency. After the user clicks the prediction button, Flask sends the values to the trained Decision Tree model and displays the predicted priority.

9.4 Project Folder Structure

```
Emergency-Complaint-Priority/
│
├── emergency_complaints_dataset.csv
├── train_model.py
├── priority_model.pkl
├── app.py
│
└── templates/
    └── index.html
```

9.5 Running the Project

33. Install the required packages using: pip install pandas scikit-learn flask
34. Keep the CSV dataset in the same folder as train_model.py.
35. Run python train_model.py to train and save the model.
36. Run python app.py to start the Flask server.
37. Open http://127.0.0.1:5000 in a web browser.
38. Enter complaint details and click Predict Complaint Priority.

10. Results

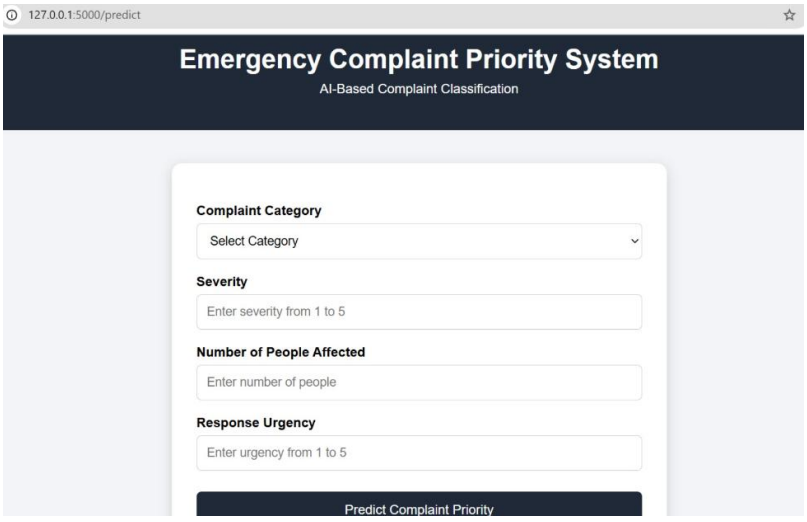
The dataset contains 300 records. The model uses an 80:20 train-test split, giving 240 training records and 60 testing records. With the supplied dataset and Decision Tree configuration (random_state=42 and max_depth=5), the measured test accuracy is 75.00%.

Performance Item	Result
Total dataset records	300
Training records	240
Testing records	60
Algorithm	Decision Tree Classifier
Test Accuracy	75.00%
Output classes	High, Low, Medium

Example Prediction Results

Category	Severity	People Affected	Response Urgency	Predicted Priority
Fire	5	25	5	High
Garbage	1	3	1	Low
Road Damage	3	12	3	Medium

The result demonstrates that the trained model can classify new complaint records into priority categories. The prediction should be treated as decision support, not as a replacement for trained human emergency personnel.



11. Advantages

39. Saves time by automatically prioritizing complaints.
40. Helps identify high-priority complaints that may require faster attention.
41. Reduces manual effort involved in sorting a large number of complaints.
42. Provides a simple and easy-to-use web interface.
43. Can support consistent classification using the same trained model.

12. Limitations

44. The model accuracy depends on the quality and size of the training dataset.
45. The dataset used in this project is synthetic and may not represent all real-world complaint situations.
46. The system should not replace human judgment in serious emergencies.
47. Only a limited number of complaint features are considered in the current version.

13. Applications

- Municipal complaint management for road, water, garbage, drainage, and electricity issues.
- College and university complaint management.
- Housing society maintenance and security complaints.
- Hospital or healthcare service complaint triage, with appropriate human oversight.
- Customer service centers for classifying urgent and routine complaints.
- Public service portals for organizing complaints according to urgency.

14. Conclusion

The Emergency Complaint Priority System was developed as an AI-based solution for classifying complaints into High, Medium, and Low priority. A synthetic dataset containing 300 records was prepared and processed using Pandas and LabelEncoder. A Decision Tree Classifier was trained using an 80:20 train-test split, and the resulting test accuracy was 75.00%.

The project demonstrates how machine learning can be integrated with a Flask web application to provide an easy-to-use prediction system. The system can support complaint management by helping staff identify which complaints may require earlier attention.

15. Future Scope

- Use a larger dataset collected from real complaint-management systems after proper anonymization and authorization.
- Add natural-language processing so users can enter the complaint in their own words instead of selecting only structured fields.
- Add location and time information to improve prioritization.
- Add an administrator dashboard for viewing, filtering, and tracking complaints.
- Connect the system with email/SMS notifications for high-priority complaints.
- Compare Decision Tree performance with Naive Bayes, kNN, and other suitable classifiers.

- Deploy the application on a secure cloud platform for authorized users.

16. References

- Python Documentation – https://docs.python.org/
- Pandas Documentation – https://pandas.pydata.org/docs/
- Scikit-learn Documentation – https://scikit-learn.org/stable/
- Flask Documentation – https://flask.palletsprojects.com/
- Scikit-learn Decision Tree User Guide – Decision Tree learning and classification.
- Project dataset: emergency_complaints_dataset.csv, synthetic academic dataset created for this project.

END OF PROJECT REPORT